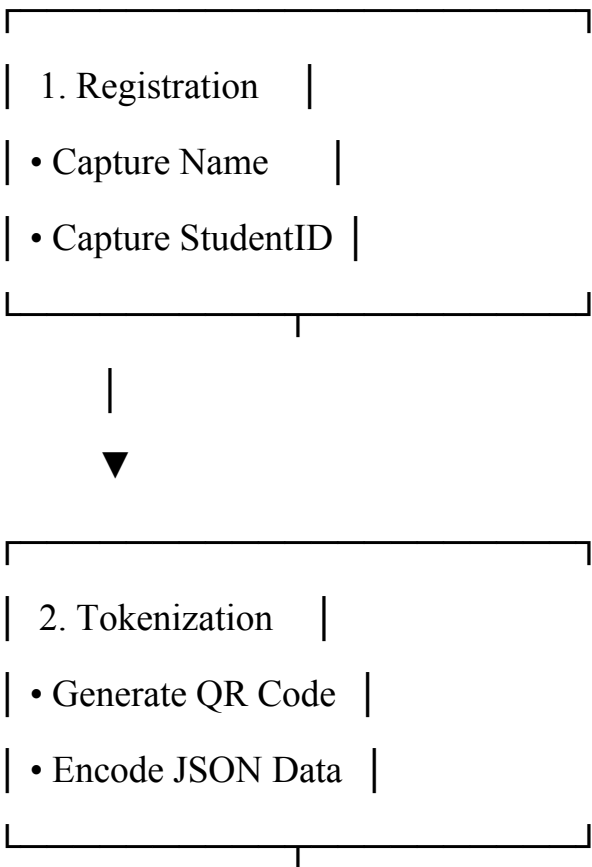


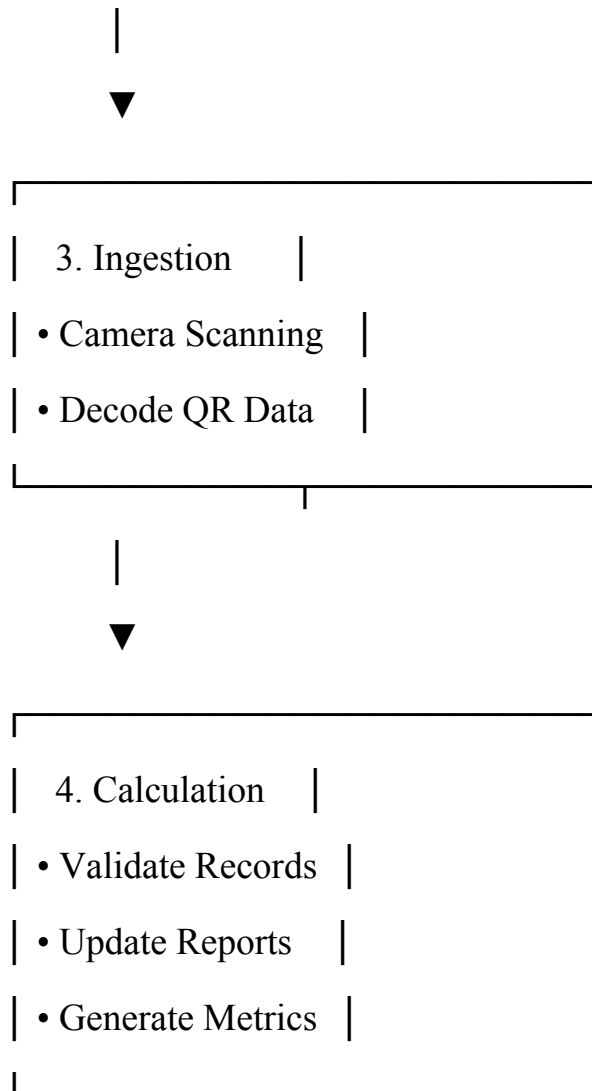
**Technical Documentation Manual: Attendance Management System
Architectural Foundation and Core Operational Flow**

The Attendance Management System application is engineered as a modern, highfidelity, clientside Single Page Application (SPA). By leveraging edge computing principles directly within the browser ecosystem, the system processes optical token checking, manages localized relational datasets, and serves advanced compliance reports completely on the user's local hardware. This architecture eliminates the need for expensive dedicated physical scanning equipment or continuous cloud computing backends.

The complete operational flow of the system works through an integrated, fourstep lifecycle:

PROSCAN ATTEND SYSTEM WORKFLOW





1. The Registration Stage

Through the user interface, the system captures identity data points (the student's full name and their unique institutional matriculation or tracking identifier). The system's form constraints sanitize these text strings on ingestion, mitigating basic clientside codeinjection and crosssite scripting (XSS) risks.

2. The Asymmetric Tokenization Stage

Once form validation succeeds, the core encoder parses the plaintext

metadata object and structures it into a strict JSON layout string. This compiled string serves as the input value for the generating engine, which dynamically maps the text bits into a canvas-rendered QR code matrix using Error Correction Level H. This high-density layout ensures that even if up to 30% of the physical token face is scratched or obscured, it remains readable.

3. The RealTime Ingestion Stage

The administrator interface controls the live camera terminal, handling continuous video stream rendering. When a user presents their credential pass to the video sensor, a background frame-capture loop isolates image buffers at a rate of 15 frames per second (15 FPS). The middleware engine extracts the raw matrix coordinates from these frames, decodes the embedded text payload back into its structured JSON format, and isolates the student identification string.

4. The Validation and Calculation Stage

Before modifying the system memory arrays, the compliance engine routes the decoded student identification string through a state-checking routine. The engine cross-references the student ID against an active chronological log array stored in the browser's persistent environment.

If a matching tracking entry already exists within the current calendar day's timestamp boundary, the transaction is rejected as an invalid proxy attempt. If the entry passes validation, a new record object is successfully committed to the database. This updates the dashboard metrics and refreshes the compliance report tables in real time.

Section 2: System Strategic Vision: Aims and Objectives

2.1 Primary System Aims

1. To eliminate systemic administrative data entry bottlenecks by replacing slow, manual, paperbased rollcall sheets and unverified shared cloud links with an automated, subsecond optical token checkin system.
2. To maximize system operational accessibility and lower deployment costs by utilizing standardized web browser frameworks, turning any common mobile device or computer webcam into an enterprisegrade tracking gate without requiring specialized biometric hardware.
3. To enforce absolute transaction integrity and nonrepudiation within classroom environments through clientside statechecking algorithms that intercept and block identity proxy manipulation patterns in real time.

2.2 Core Technical Objectives

Objective 1 (Optical Framework Engineering): Build an edgebased processing pipeline that successfully initializes local camera media devices via native hardware APIs (`getUserMedia()`), sampling frames at an optimal performance rate to decode encrypted string matrices within 350 ms of card presentation.

Objective 2 (Advanced Statistical Core Implementation): Design an automated reporting engine that references realtime tracking arrays against a variable term session parameter to compute individual attendance percentages, dynamically sorting student profiles into categorized risk alerts (Good Standing, Low Margin, Critical Risk).

Objective 3 (Data Serialization and MultiFormat Portability): Engineer a clientside document compiler capable of structuring tabular tracking datasets into clean data structures, allowing users to export documents natively (.csv, .xls, .doc, .pdf) directly from the client interface.

Section 3: Deep Technical JavaScript Implementation Analysis

The operational logic of Attendance Management System is driven by functional scripts engineered using modern native ECMAScript (\$ES6+\$) concepts. The application code is separated into four key functional blocks:

3.1 Global State Structures and Event Life Hooks

```
javascript
let students = JSON.parse(localStorage.getItem("ps_students")) || [];
let attendanceLogs = JSON.parse(localStorage.getItem("ps_logs")) || [];
let activeTab = "dashboard";
let html5QrcodeScanner = null;
```

The application establishes global state parameters to maintain realtime data consistency across different views. The systems' state arrays (students and attendanceLogs) use synchronous data parsing to sync directly with the sandboxed localStorage object container. This design guarantees complete offline performance and total data privacy since no sensitive tracking datasets are transmitted over external networks.

3.2 The Tokenizer Logic (QR Generation Engine)

```
javascript
const payload = JSON.stringify({ id, name });
new QRCode(qrContainer, {
  text: payload,
  width: 160,
  height: 160,
  colorDark: "0f172a",
  colorLight: "ffffff",
  correctLevel: QRCode.CorrectLevel.H
});
```

When the student registration form is submitted, the system halts standard browser page refreshes via an event listener interceptor (`e.preventDefault()`). The handler groups the sanitized inputs into a compact JSON configuration payload string. This structured payload string is injected into the thirdparty QRCode constructor core, which uses a numeric matrix allocation algorithm to convert the data characters into vector squares rendered on a clean HTML5 canvas display container.

3.3 The AntiProxy Verification Core (Live Processing Engine)

```
javascript
const todayStr = new Date().toDateString();
const isDoubleDip = attendanceLogs.some(log => log.id === id && new
Date(log.timestamp).toDateString() === todayStr);

if (isDoubleDip) {
  showToast(Duplicate entry intercepted for ID: ${id}, "error");
  return "duplicate";
}
```

The application's core data protection mechanism relies on a functional array matching check (`.some()`). When the camera terminal reads an incoming identity string, the antiproxy verification core checks the active transaction ledger for previous entries matching that unique student ID.

By mapping the evaluation criteria directly to the localized calendar date string (`toDateString()`), the system creates a secure daylong restriction window. This prevents users from executing duplicate checkins within the same tracking session.

3.4 Automated Statistical Reports Engine

javascript

```
const attended = attendanceLogs.filter(log => log.id === student.id).length;  
const absent = Math.max(0, totalSessions - attended);  
const percentage = Math.min(((attended / totalSessions) * 100),  
100).toFixed(1);
```

The analytics engine reads tracking logs using a conditional loop filter (.filter()) to isolate all logged instances matching a specific student's unique identifier. The resulting array count determines the total attended sessions.

The system references this value against the global parameter (totalSessions) to calculate individual tracking percentages using standard mathematical mapping:

$$\text{Compliance Rate \%} = \left(\frac{\text{Attended Logs Count}}{\text{Configured Total Sessions Limit}} \right) \times 100\%$$

The engine then uses conditional branching to dynamically assign style properties to the UI reporting elements based on the resulting percentage calculation:

```
If Compliance >= 75%    > Assign "badgesuccess" [Good Standing]  
Else If Compliance >= 50% > Assign "badgewarning" [Low Margin]  
Else                    > Assign "badgedanger" [Critical Risk]
```

Section 4: Testing and System Validation Protocols

To verify that the system is stable and ready for deployment, use these manual testing protocols to evaluate user workflow modules, platform performance limits, and security boundaries.

4.1 System Setup and Execution Environment

1. Save the codebase elements into separate files (index.html, style.css, and script.js) inside a single directory folder.
2. Launch a secure testing container environment. This can be done by using a local development server plugin (such as VS Code's Live Server) or by opening the index file within a web browser that supports media capture integrations (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge).
3. Ensure that an integrated web camera or an external USB hardware video capture device is connected and authorized within your operating system's device manager settings.

4.2 Functional Test Case Suite

Test Case 1: Student Identity Registration & Pass Rendering

Execution Strategy: Open the interface, navigate to the Dashboard tab view, and locate the form panel labeled Issue Student QR Pass. Fill the inputs with test data strings (e.g., Full Name: John Doe, Student ID: MAT2026001), and click the Register & Generate Pass action button.

Expected Verification Standard: The form should clear cleanly, the global Total Enrolled analytics counter badge should instantly increment to 1, and a highcontrast vector QR code should display on the card interface with a readable label matching your input data.

Test Case 2: Terminal Ingestion and CheckIn Tracking

Execution Strategy: Navigate to the Live Terminal section via the hamburger menu toggle. Grant the browser application permission to access

your webcam device. Once the video feed initializes, hold up a mobile screen displaying the generated QR pass or present a clear printout of the token to the camera frame.

Expected Verification Standard: The parsing middleware should process the optical data within a subsecond interval. The status feedback container should switch to a bright green background displaying an Access Granted success message along with the student's name and ID.

Test Case 3: AntiProxy Guard Interception

Execution Strategy: Without reloading the tracking interface, present the same student QR pass token to the video sensor a second time within the active tracking window.

Expected Verification Standard: The verification engine should catch the matching identifier record. The system must reject the transaction, trigger a flashing red Duplicate Entry error alert in the status feedback box, and play an alert notification while keeping the attendance logs unchanged.

Test Case 4: Contextual Compliance Metrics Update

Execution Strategy: Navigate to the Attendance Reports panel via the main sidebar. Confirm that the test student's summary row appears in the table with an accurate tracking percentage (e.g., 1 session out of a total session count of 45 should render exactly as \$2.2\%\$, flagged with a red Critical Risk warning badge). Next, navigate to the top header bar and adjust the Total Term Sessions input configuration parameter from 45 down to 1. Return to the Attendance Reports screen.

Expected Verification Standard: The reporting engine must recalculate the student's metrics instantly. The progress indicator should update to \$100.0\%\$, the color property should switch to green, and the status label should automatically upgrade to a Good Standing classification status badge.

Test Case 5: Document Engine Data Exporting

Execution Strategy: While viewing the Attendance Reports section, click

the Export Report selection element block. Choose Microsoft Excel (.xlsx) or PDF Document (.pdf) from the option layout list, then click the Export Report trigger button.

Expected Verification Standard: The document compiler should run without external network lag. The browser will natively trigger a direct download popup offering a formatted document that contains your exact local registration records.